جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 12: Markov Decision Processes - Decision Theory

Prof. Anis Koubaa

akoubaa@alfaisal.edu

Welcome to our exploration of decision-making under **uncertainty**. Today we'll move beyond deterministic search problems to tackle real-world scenarios where outcomes are **probabilistic** 🎲





The Challenge: Navigating Uncertainty in Riyadh

You're at Alfaisal University, needing to reach King Abdullah Financial District (KAFD) quickly. What's your best option?

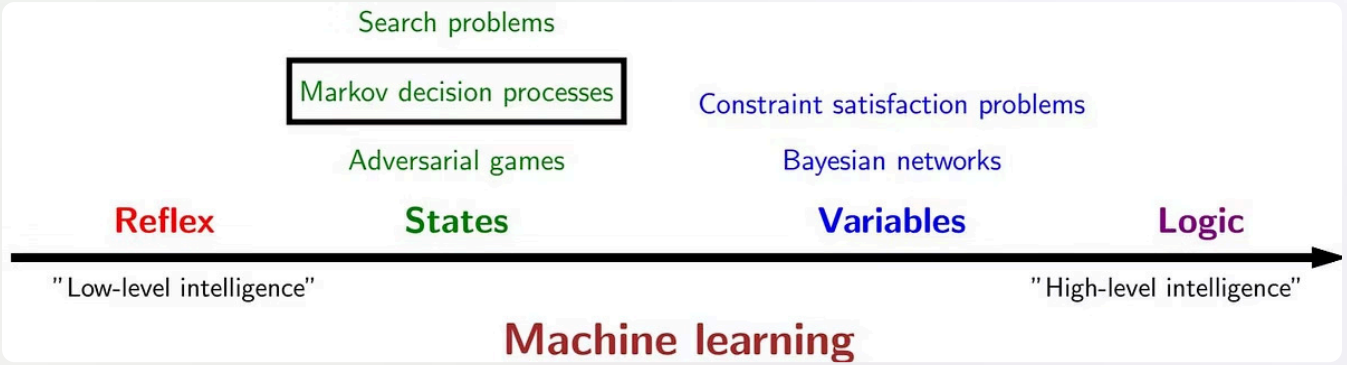
🚴 Bike/Scooter | 🚗 Drive | 🚆 Riyadh Metro

🚕 Careem | ✈️ Bus

This seemingly simple choice is full of uncertainty:

- **Traffic fluctuations** on major Riyadh roads
- **Surge pricing** on Careem/Uber
- **Metro delays** (once operational)
- **Weather impacts** like rain or sandstorms
- **Unexpected road closures or events**

⚠ **How can we make the optimal choice when faced with such probabilistic outcomes?**



Course Roadmap 🗺️

Low-Level Intelligence

- **Search problems**
- **Markov decision processes** ← We are here
- Adversarial games

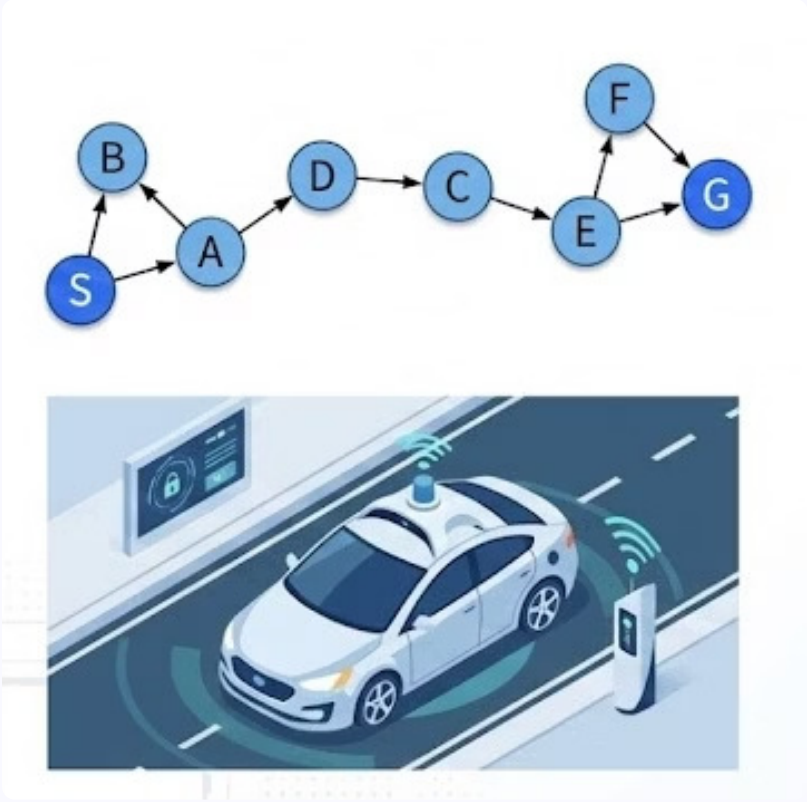
States: How things are

High-Level Intelligence

- Constraint satisfaction
- Bayesian networks
- Machine learning

Variables & Logic: What things mean

Recap: Search Problems



We studied **search problems**–a powerful paradigm for solving diverse challenges:

- Package delivery 📦
- Route finding 🗺️

Key assumption: Actions lead **deterministically** to unique successor states

$$\text{Succ}(s, a) \to s'$$

MDP as the Probabilistic Version of Traditional Search

MDP extends classical search to uncertainty.

1 Traditional Search

Each action leads to one deterministic successor state.

Example: `A -> B` (always)

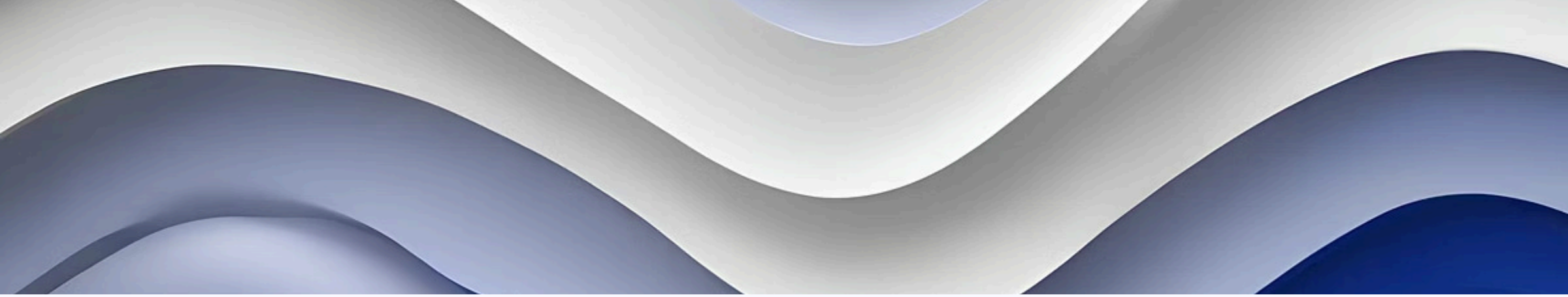
2 MDP (Markov Decision Process)

Each action leads to a probability distribution over next states.

Example: `A -> B` with 0.7, `A -> C` with 0.3

✔️ **Search under certainty** → deterministic transitions

✔️ **MDP** → search under **uncertainty** (probabilistic transitions)



The Challenge: Real-World Uncertainty 🌪️

Randomness Everywhere

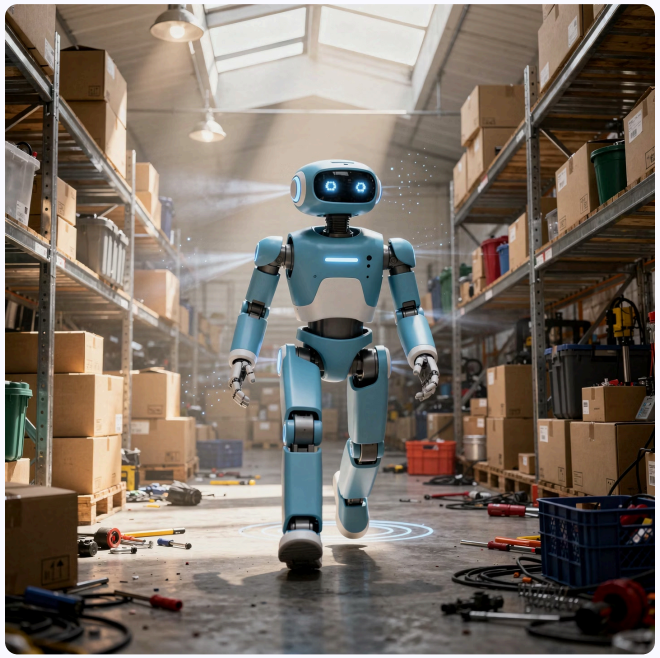
Real systems involve **stochastic outcomes**: sensor noise, unpredictable events, environmental changes

The Deep Question

How can we act **optimally** when outcomes are uncertain? A single deterministic plan won't suffice

✓ **Today's goal:** Develop mathematical tools to handle **probabilistic transitions** while preserving the intuitive notion of **states** from search problems.

Real-World Applications 🌍



Robotics: Navigate environments where **actuators fail**, obstacles appear unexpectedly, and sensors provide noisy measurements



Resource Allocation: Decide production schedules without knowing future **customer demand** for various products



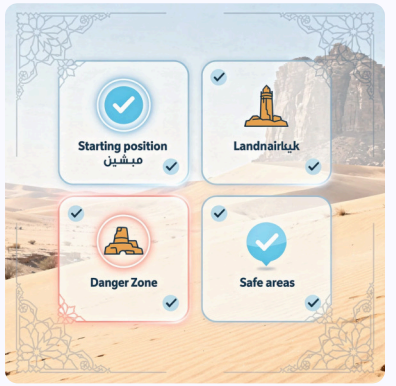
Agriculture: Choose crops to plant without knowing future **weather patterns** and resulting yields

All these scenarios share a common thread: **uncertainty from sensors, markets, or nature** can be handled in a unified mathematical framework.



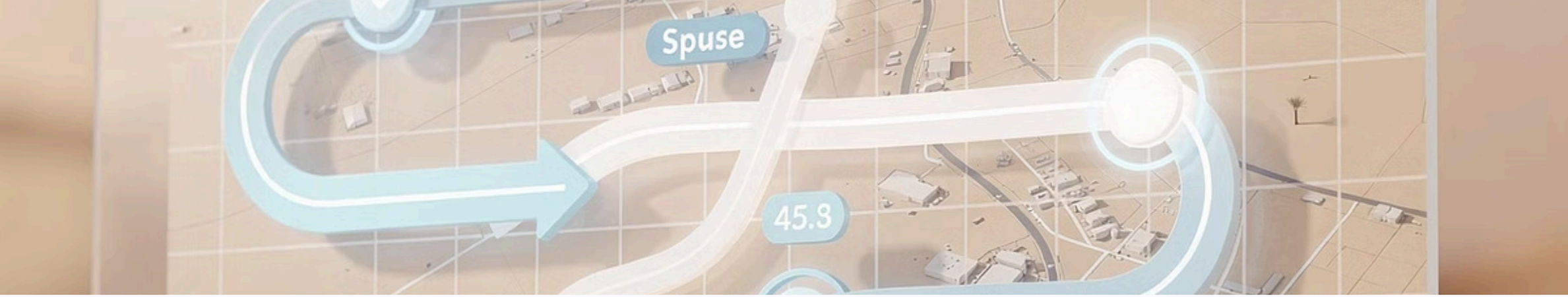
Running Example: Desert Navigation 🏜️

You're navigating the Saudi desert modeled as a **3×4 grid**. From each state, take actions **N, E, S, W** to move to adjacent cells.



- 🏠 **Start:** Position (2,1)
- 🔥 **Extreme heat zone:** Reward = -50
- 🕌 **Historic landmark:** Reward = 20
- 🌴 **Oasis area:** Reward = 2

Attempting to move off the grid keeps you in place. The challenge: **desert winds are unpredictable**—there's a probability you'll be **blown in a random direction**! 😱



Interactive Demo: Desert Navigation Dynamics

Each state shows its **maximum expected utility** (analogous to FutureCost in search). Arrows indicate the **best action** from each cell.

01

Deterministic case (windProb = 0)

Optimal path: Go directly for the historic landmark (reward 20)

02

Low uncertainty (windProb = 0.1)

Still worth risking the path to reward 20

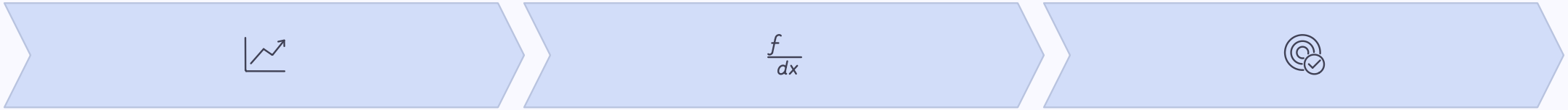
03

High uncertainty (windProb = 0.3)

Too risky! Better to play it safe and aim for the oasis (reward 2)

Key insight: We specify only the **world dynamics**, not the best action. Optimal actions emerge automatically from our algorithms! 🎯

Today's Roadmap 📋



Markov Decision Processes

Formal framework for decision-making under uncertainty

Policy Evaluation

Computing the value of a given strategy

Value Iteration

Finding the optimal strategy



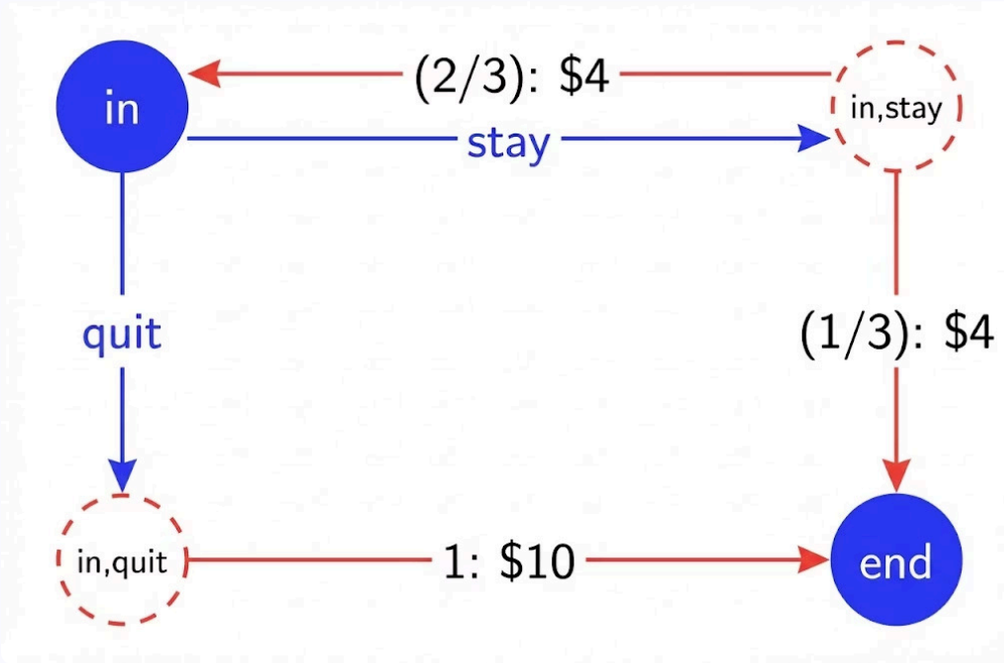
Motivating Example: The Dice Game 🎲

Game Rules

For each round $r = 1, 2, \ldots$

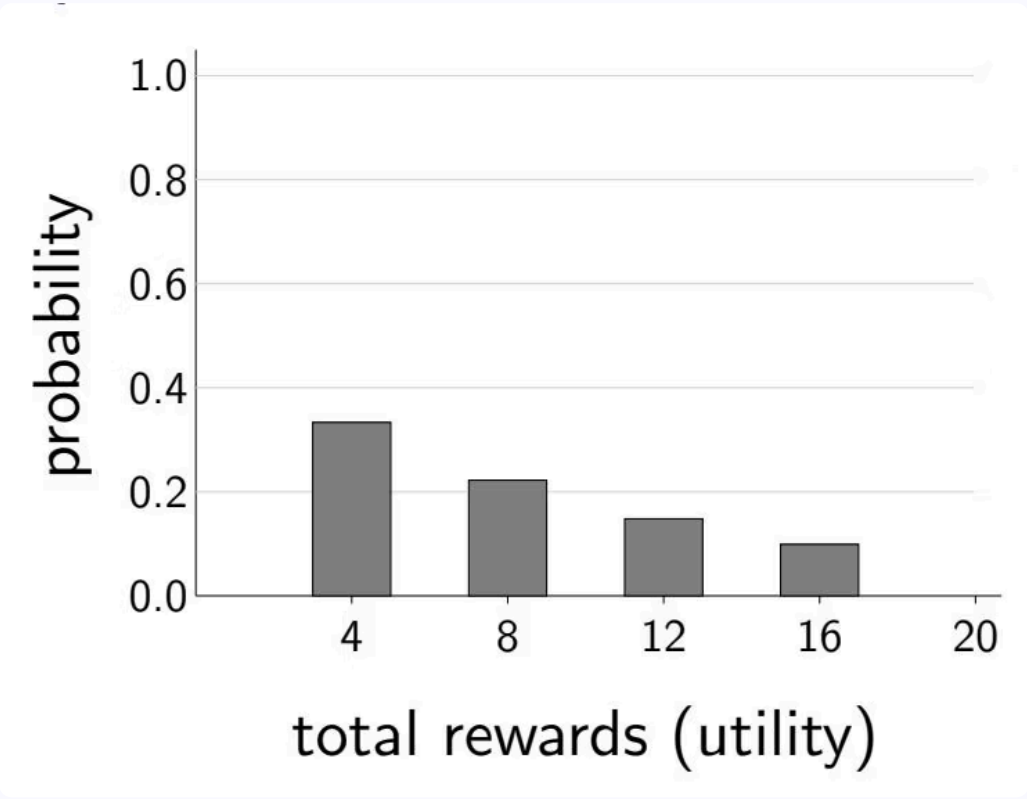
1. You choose **stay** or **quit**
2. If **quit**: receive $10 and game ends
3. If **stay**: receive $4, then roll a 6-sided die
   - Roll 1 or 2 → game ends
   - Roll 3-6 → continue to next round

Question: What's the optimal strategy? Should you stay or quit? 🤔



Strategy Analysis: Always Stay

Possible Outcomes



Each game produces a different sequence of rewards, resulting in a **utility** (sum of rewards)

Expected Utility Calculation

$$\frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) + \cdots = 12$$

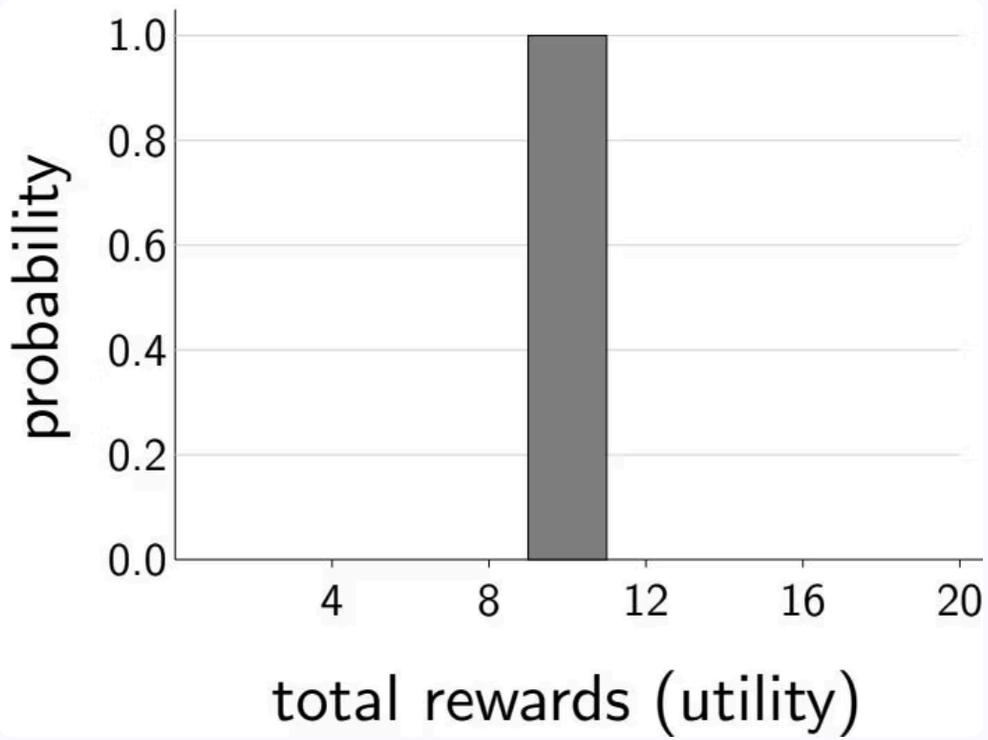
With probability $\frac{1}{3}$, game ends after round 1 (utility = 4)

With probability $\frac{2}{3} \cdot \frac{1}{3}$, game ends after round 2 (utility = 8)

And so on...

Strategy Analysis: Always Quit

Deterministic Outcome



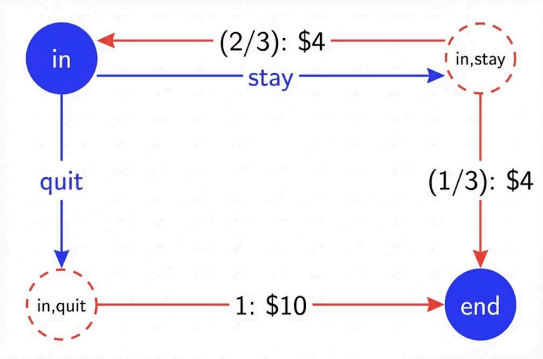
Quit immediately → guaranteed reward of $10

Expected Utility

$$1 \cdot (10) = 10$$

No randomness here! Utility is always 10.

🏆 **Verdict:** "Stay" strategy has **expected utility = 12**, beating "quit" strategy (expected utility = 10). Even though some outcomes yield less than 10, staying is optimal *on average*.

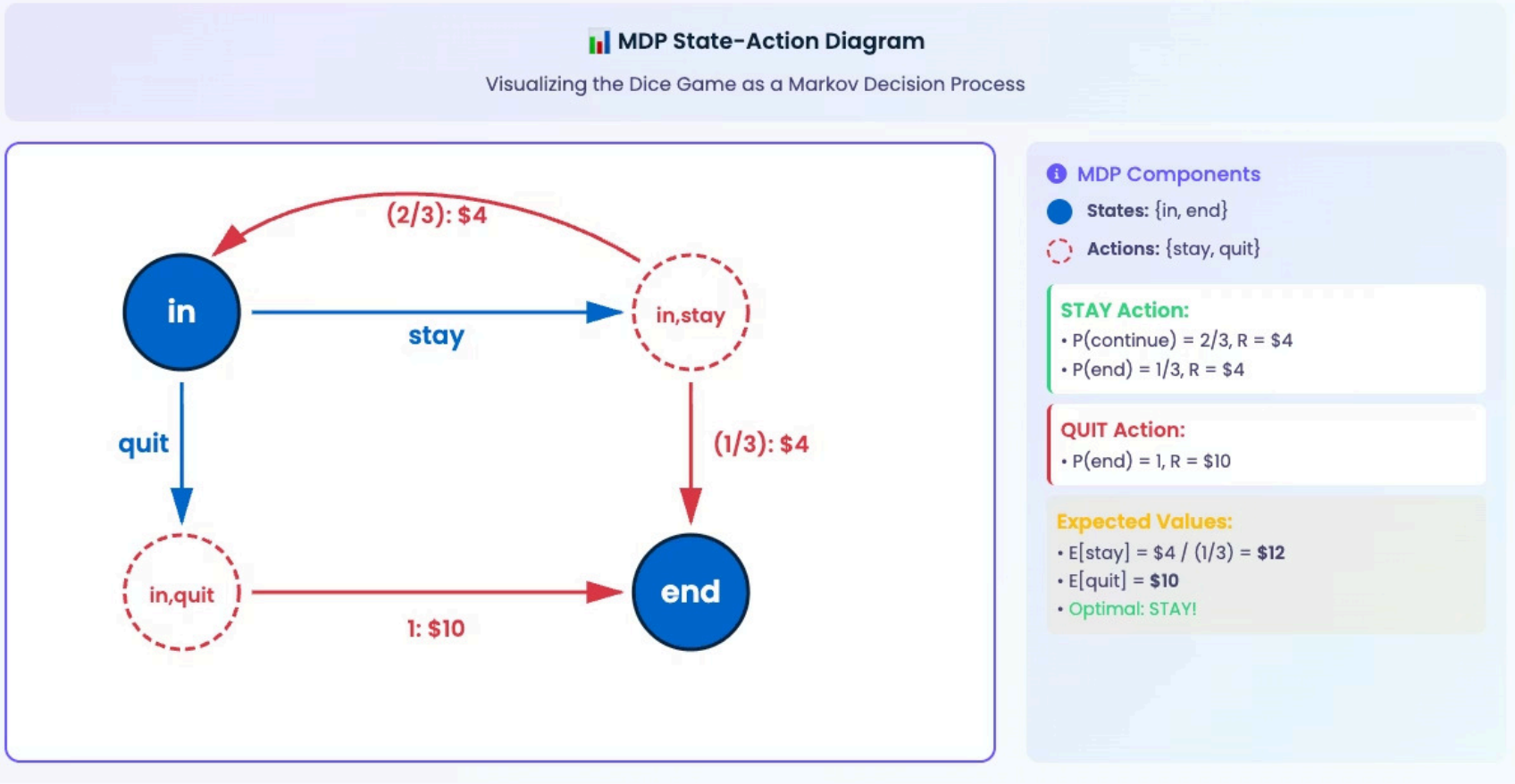


Formalizing the Dice Game as an MDP

We need a more **general framework** to handle the dice game, volcano crossing, and beyond. Enter: **Markov Decision Processes** 🎯

States (circles) Represent situations: "in", "end"	**Actions (edges)** Choices available: "stay", "quit"
Chance Nodes Probabilistic outcomes after actions	**Transitions** Probabilities and rewards for each outcome

MDP State-Action Diagram



MDP Graph Representation 📊

Graph Structure

- **Nodes:** States + chance nodes
- **State edges:** Possible actions
- **Chance edges:** Random outcomes (with probabilities + rewards)

Reading the Graph

From state "in", action "stay" leads to a chance node, which transitions to:

- "in" with prob $\frac{2}{3}$, reward 4
- "end" with prob $\frac{1}{3}$, reward 4

Formal Definition: Markov Decision Process

An MDP is a 7-tuple $(S, s_{\text{start}}, A, T, R, \text{IsEnd}, \gamma)$ consisting of:

- **States**: S - The set of all possible states.
- **Starting State**: $s_{\text{start}} \in S$ - The **initial state** where the decision process begins.
- **Actions**: $A(s)$ - The set of possible actions available from state s.
- **Transition Probability**: $T(s, a, s')$ - The probability of reaching state s' after taking action a in state s.
- **Reward Function**: $R(s, a, s')$ - The immediate reward received when transitioning from s to s' via action a.
- **Terminal States**: $\text{IsEnd}(s)$ - A predicate indicating whether s is a terminal (end) state.
- **Discount Factor**: γ - A value $0 \leq \gamma \leq 1$ (default = 1) that weighs the importance of future rewards.

Formal Definition: Markov Decision Process

1

States

Set of all possible states: States

Starting state: $s_{\text{start}} \in \text{States}$

2

Actions & Transitions

$\text{Actions}(s)$: possible actions from state s

$T(s, a, s')$: probability of reaching s' after action a from state s

3

Rewards & Termination

$\text{Reward}(s, a, s')$: reward for transition (s, a, s')

$\text{IsEnd}(s)$: whether s is a terminal state

4

Discount Factor

$0 \leq \gamma \leq 1$: how much we value future rewards (default: 1)

Key Property: Transition Probabilities

The transition probabilities $T(s, a, s0)$ specify the probability of ending up in **state** s' if taken **action** a in **state** s.

Probability Distribution

For each state s and action a:

$$\sum_{s' \in \text{States}} T(s, a, s') = 1$$

Successors: States s' where $T(s, a, s') > 0$

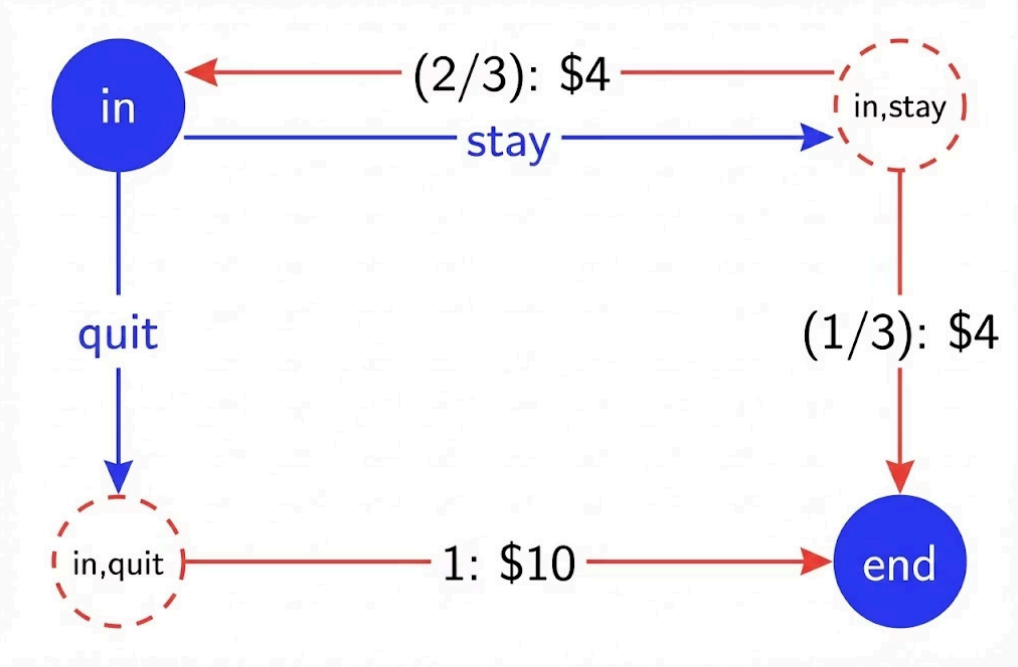
Example: Dice Game

s	a	s'	$T(s, a, s')$
in	quit	end	1
in	stay	in	$\frac{2}{3}$
in	stay	end	$\frac{1}{3}$

Key Property: Transition Probabilities

Transition Probabilities

s	a	s'	$T(s, a, s')$
in	quit	end	1
in	stay	in	$\frac{2}{3}$
in	stay	end	$\frac{1}{3}$



Traditional Search Problems

Definition: Search Problem

- **States**: the set of states
- $sstart \in Statess_{\text{start}} \in \text{States}$**sstart∈States**: starting state
- **Actions(s)**: possible actions from state **s**
- **Succ(s, a)**: the state we end up in after taking action **a** in state **s**
- **Cost(s, a)**: the cost of taking action **a** in state **s**
- **IsEnd(s)**: whether **s** is an end (terminal) state

Relation to MDPs

- **Succ(s, a)** ⇒ corresponds to $T(s, a, s\prime)T(s, a, s')$**T(s,a,s′)**
- **Cost(s, a)** ⇒ corresponds to **Reward(s, a, s′)**

MDPs vs. Search Problems: Key Differences

Search Problems

- **Deterministic:** $\text{Succ}(s, a)$ → unique next state
- **Minimize:** Path cost
- **Solution:** A sequence of actions (path)

Markov Decision Processes

- **Stochastic:** $T(s, a, s')$ → probability distribution over next states
- **Maximize:** Expected rewards
- **Solution:** A policy (action for each state)

MDPs generalize search problems: Set $T(s, a, s') = 1$ if $s' = \text{Succ}(s, a)$, else 0.

What Is a Policy ? 🎯

Think about a **policy** as a **possible search solution**

Definition

A **policy** π is a function that maps each state to an action:

$$\pi : \text{States} \rightarrow \text{Actions}$$

Tells agent ***what*** *to do* in **every** possible situation

Example: Dice Game

- $\pi_1(\text{in}) = \text{quit}$
- $\pi_2(\text{in}) = \text{stay}$

Different policies lead to different expected outcomes!

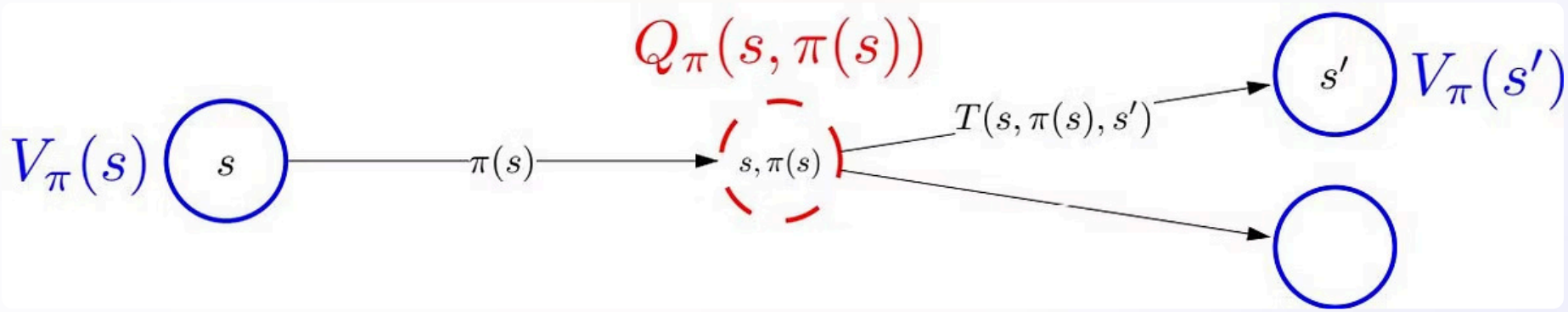
Unlike search problems (which output a **path**), MDPs produce **policies** that handle all contingencies. 🛡️



Policy Evaluation

Now that we've defined MDPs (the *input*) and policies (the *output*), we need a way to **evaluate** how good a policy is. 📊





Value and Q-Value: Core Concepts

Value Function $V_\pi(s)$

Expected utility obtained by following policy π starting from state s

Answers: "How good is this state under policy π?"

Q-Value Function $Q_\pi(s, a)$

Expected utility of taking action a from state s, then following policy π

Answers: "How good is this action in this state?"

The Value-Q-Value Recurrence 🔄

From Value to Q-Value

If not an end state:

$$V_\pi(s) = Q_\pi(s, \pi(s))$$

Value = Q-value of the action specified by policy

Base case: $V_\pi(s) = 0$ if $\text{IsEnd}(s)$

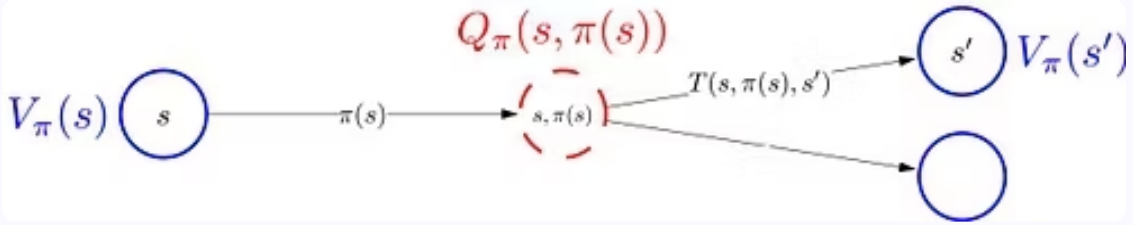
From Q-Value to Value

$$Q_\pi(s, a) = \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_\pi(s')]$$

Immediate reward + discounted future value

The **Q** stands for **"Quality"** of an action.

These recurrences form the backbone of policy evaluation! They express **expected utility recursively**. 💡



Q-value Definition

The **Q** stands for **“Quality”** of an action.

So a **Q-value** means:

> **The quality of taking action *a* in state *s* and how good the future return is.**

Formally:

$$Q_\pi(s,a) = \sum_{s'} T(s,a,s')[\text{Reward}(s,a,s') + \gamma V_\pi(s')]$$

🧠 Intuition

- **V(s)** = value of being in a state
- **Q(s,a)** = value (quality) of taking a specific action *from* that state

So **V** evaluates states,
and **Q** evaluates **state-action pairs**.

✅ It tells us **how good** (quality) an action is before we actually take it.

The Bellman Equation: Core of MDPs

The **Bellman Equation** is the fundamental recursive equation that defines the value of a state in an MDP.

It states:
The value of a state = immediate reward + discounted value of the next state.

Bellman Equation for a Policy (π)

This equation defines the expected value of a state s when following a specific policy π.

$$V_\pi(s) = \sum_{s'} T(s, \pi(s), s') \Big[\text{Reward}(s, \pi(s), s') + \gamma V_\pi(s') \Big]$$

It tells you the value you can expect if you consistently follow policy π from state s.

Bellman Optimality Equation

This equation defines the maximum possible value achievable from a state s by choosing the optimal action.

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') \Big[\text{Reward}(s, a, s') + \gamma V^*(s') \Big]$$

It reveals the true optimal value of a state, assuming you always pick the best action at every step.

Value and Q-Value Under a Policy π

These equations formally define the relationships between the state value function and the Q-value function, which are crucial for evaluating any given policy π.

State Value Function $V_\pi(s)$

$$V_\pi(s) = \begin{cases} 0 & \text{if IsEnd}(s), [6pt] Q_\pi(s, \pi(s)) \ \text{otherwise.} \end{cases}$$

This function calculates the total expected reward an agent can accumulate starting from state s and subsequently adhering to policy π.

Q-Value Function $Q_\pi(s, a)$

$$Q_\pi(s, a) = \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_\pi(s')]$$

The Q-value measures the expected reward obtained by taking a specific action a from state s, and then continuing to follow policy π thereafter.

Policy Evaluation Algorithm

1

Initialize

Set $V_\pi^{(0)}(s) \leftarrow 0$ for all states s

2

Iterate (Bellman Equation)

For $t = 1, \ldots, t_{\text{PE}}$, update each state:

$$V_\pi^{(t)}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[\text{Reward}(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s')]$$

3

Converge

Stop when values stabilize: $\max_s |V_\pi^{(t)}(s) - V_\pi^{(t-1)}(s)| \leq \epsilon$

Key idea: Start with arbitrary values, repeatedly apply recurrences until convergence. ✨

Stochastic Policies

Unlike deterministic policies that prescribe a single action for each state, a **stochastic policy** introduces an element of randomness.

Definition

A **stochastic policy** specifies a probability distribution over actions for each state. Instead of picking one action, it assigns a probability to each available action:

$$\pi(a \mid s) = P(\text{action } a \mid \text{state } s)$$

This means the agent chooses action a from state s with probability $\pi(a \mid s)$.

Example: Dice Game

For our dice game, a stochastic policy in the "in" state might be:

- "Quit with probability 0.2, Stay with probability 0.8"
- "Quit half the time, stay half the time"

Formally, for any probability $p \in [0, 1]$:

$$\pi_p(\text{in}) = \begin{cases} \text{stay} & \text{with prob } 1 - p \\ \text{quit} & \text{with prob } p \end{cases}$$

There are infinitely many such policies, as p can be any value between 0 and 1.

Implementation Details 🛠️



Convergence Criterion

Repeat until maximum change across all states falls below threshold ϵ

$$\max_{s \in \text{States}} |V_\pi^{(t)}(s) - V_\pi^{(t-1)}(s)| \leq \epsilon$$



Memory Efficiency

Only store last two iterations: $V_\pi^{(t)}$ and $V_\pi^{(t-1)}$

No need to keep entire history!



Runtime Complexity

For each of t_{PE} iterations, loop through S states and their successors

$$O(t_{\text{PE}} \cdot S \cdot S')$$



Policy Evaluation in Action: Dice Game

Let's evaluate the "stay" policy: $\pi(\text{in}) = \text{stay}$

1 **Iteration 0**

$V_\pi^{(0)}(\text{in}) = 0$

2 **Iteration 1**

$V_\pi^{(1)}(\text{in}) = \frac{2}{3}(4+0) + \frac{1}{3}(4+0) = 4$

3 **Iteration 2**

$V_\pi^{(2)}(\text{in}) = \frac{2}{3}(4+4) + \frac{1}{3}(4+0) \approx 6.67$

4 **Convergence**

$V_\pi(\text{in}) \to 12$ ✓

Values converge **rapidly** to the true expected utility! 🚀

Summary: Policy Evaluation ✅

> Given an MDP and a policy π, policy evaluation computes $V_\pi(s)$–the **expected utility** of following π from each state s.

- Write recurrences relating value and Q-value
- Turn recurrences into iterative algorithm
- Initialize arbitrarily and iterate until convergence

But we still need to *find* the best policy! Next up: **Value Iteration**. 🎯



Value Iteration

We can evaluate any given policy, but how do we **find the optimal policy**? 🏆

The Goal: Optimal Value

Definition

The **optimal value** $V_{\text{opt}}(s)$ is the **maximum value** attained by *any* policy:

$$V_{\text{opt}}(s) = \max_{\pi} V_{\pi}(s)$$

Intuition

If we play the game perfectly, what's the best expected outcome we can achieve from state s?

Instead of fixing a policy π, we'll optimize *over all policies* to find V_{opt} and π_{opt}! 💪

Optimal Value Recurrence

From Value to Q-Value

If not an end state:

$$V_{\text{opt}}(s) = \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

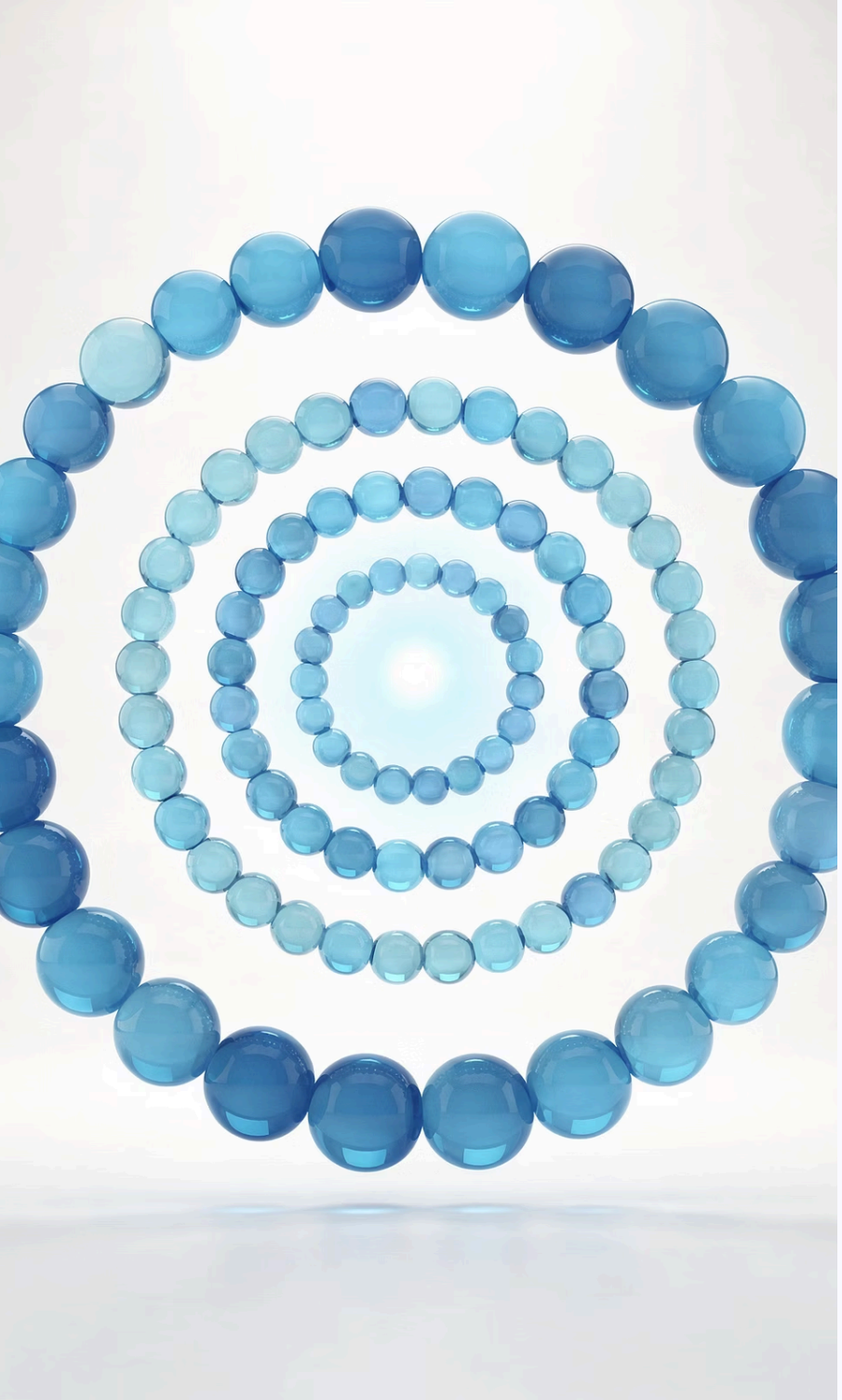
Take the **best action** (not a fixed policy action!)

From Q-Value to Value

$$Q_{\text{opt}}(s, a) = \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_{\text{opt}}(s')]$$

Same as before: immediate reward + discounted future

The key difference: $\max$ instead of following a fixed policy! This one change makes all the difference. 🌟



Value Iteration Algorithm

Initialize

Set $V_{\text{opt}}^{(0)}(s) \leftarrow 0$ for all states

Iterate

For $t = 1, \ldots, t_{\text{VI}}$, update each state:

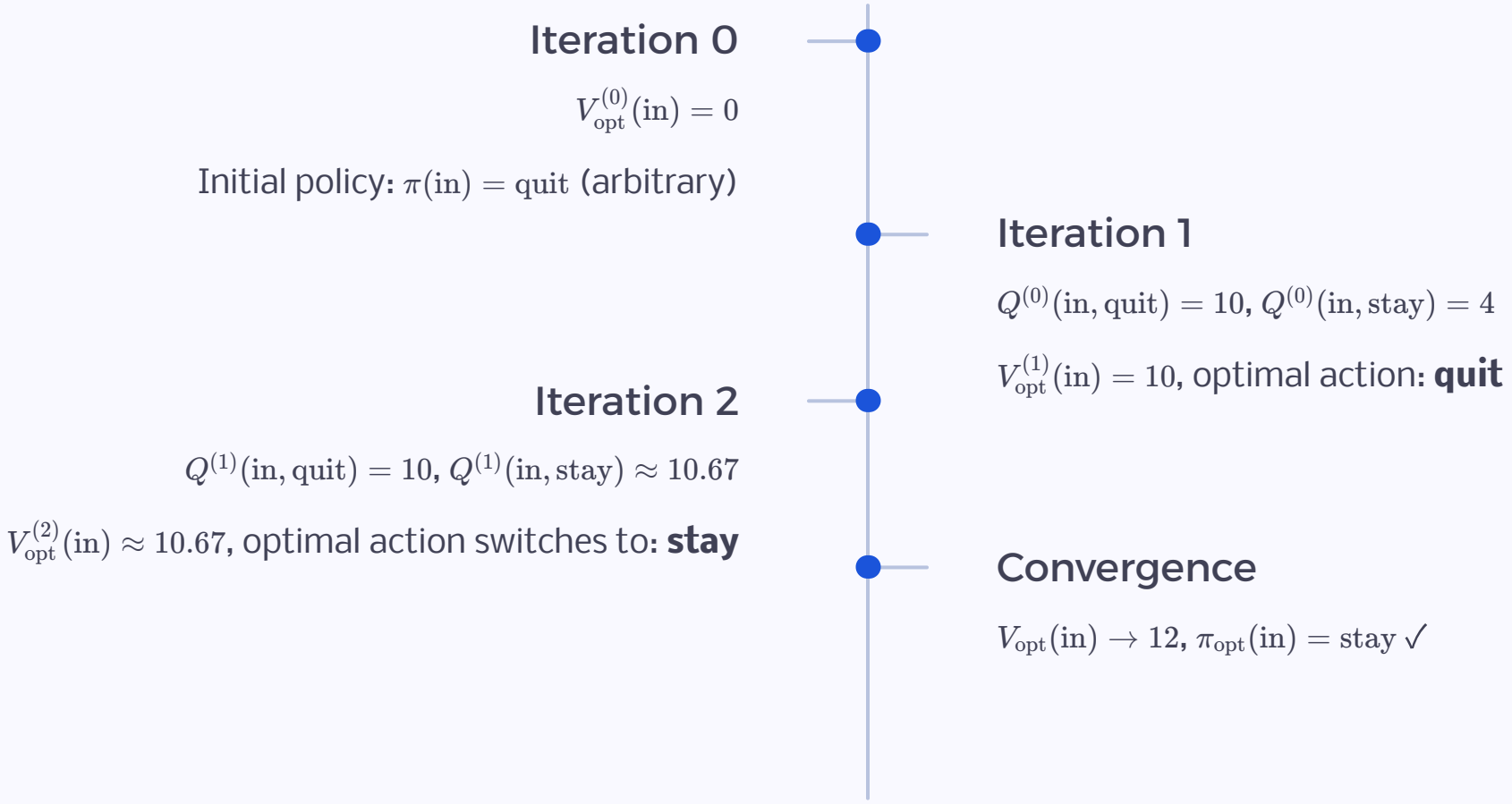
$$V_{\text{opt}}^{(t)}(s) \leftarrow \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s')[\text{Reward}(s, a, s') + \gamma V_{\text{opt}}^{(t-1)}(s')]$$

Extract Policy

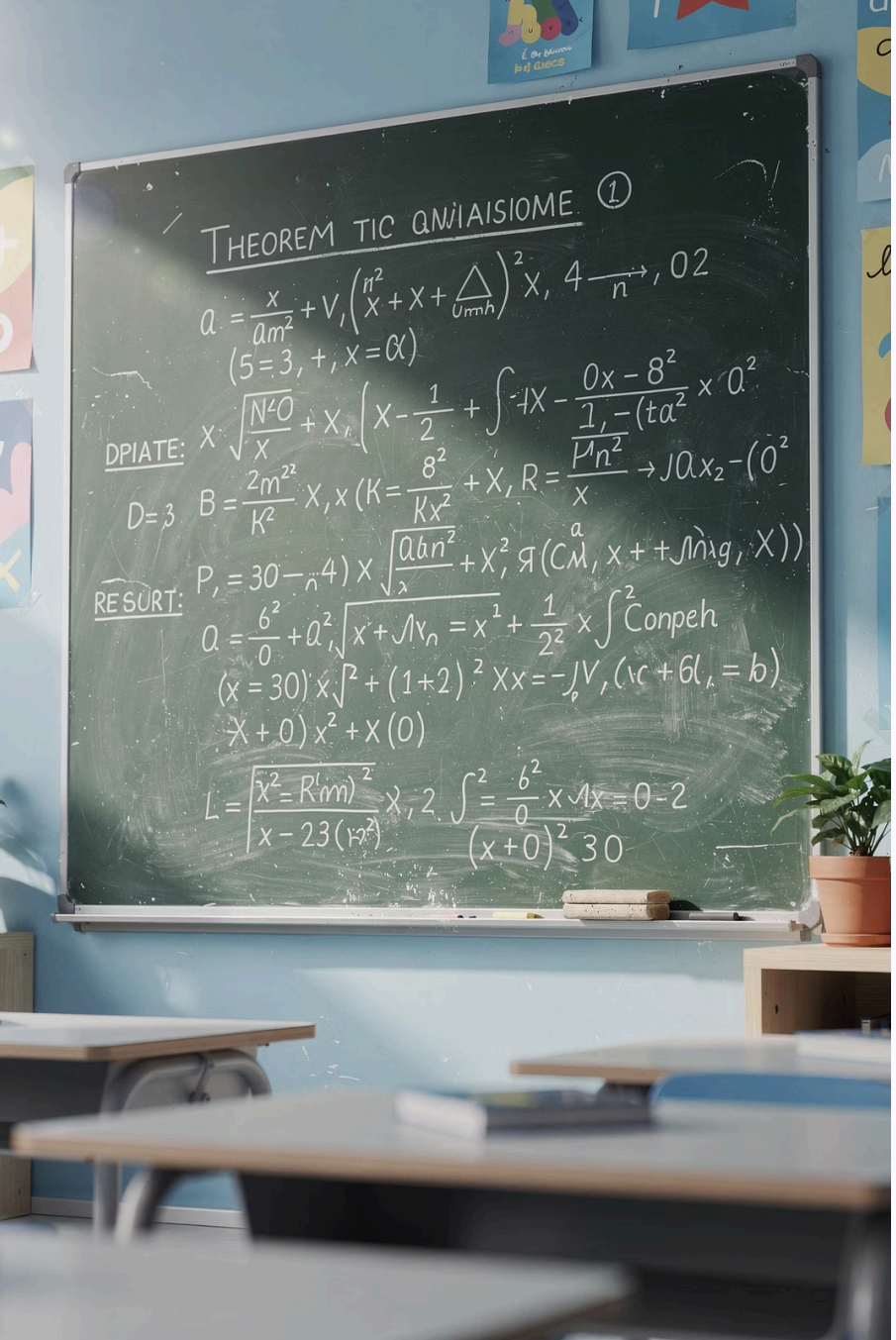
$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

Runtime: $O(t_{\text{VI}} \cdot S \cdot A \cdot S')$ (now depends on number of actions A!)

Value Iteration: Dice Game Example



Notice how the optimal policy *emerges* as values propagate! 🌱



When Do These Algorithms Work? 🤔

> ### Convergence Guarantee
>
> Policy evaluation and value iteration **converge** to the correct values if:
>
> - The discount factor $\gamma < 1$, **OR**
> - The MDP graph is **acyclic**

Intuition: Either future rewards matter less ($\gamma < 1$), or all paths eventually terminate (acyclic). Both ensure finite expected utility. ✅

Discount Factor as Termination Probability

Alternative View

Setting $\gamma < 1$ is equivalent to:

- Adding a transition to a special "end" state with probability $1 - \gamma$ from every state
- Multiplying all other transitions by γ
- Setting actual discount to 1

Interpretation

With probability $1 - \gamma$, the MDP **terminates randomly** at any step

Ensures all sampled paths are **finite with probability 1**

Without these conditions, infinite loops can cause algorithms to fail! 🚫

Counterexample: Cyclic Graph with $\gamma = 1$

Consider an MDP with two states forming a cycle, where every action moves to the other state with reward 1:

$$V_{\text{opt}}(s_1) = 1 + V_{\text{opt}}(s_2), \quad V_{\text{opt}}(s_2) = 1 + V_{\text{opt}}(s_1)$$

Value iteration never changes from initialization! All paths have **infinite length**, so expected utility is undefined. 💥

Lesson: Ensure discount $\gamma < 1$ or acyclicity for meaningful results.

Algorithm Summary 📝

Policy Evaluation

Input: MDP + policy π

Output: V_π (value of π)

Evaluates a *given* strategy

Value Iteration

Input: MDP

Output: V_{opt} + π_{opt}

Finds the *optimal* strategy

Both algorithms follow the same recipe: write recurrences, then iterate! 🔄

The Unifying Recipe 👨‍🍳

01

Write down recurrences

Express desired quantity (e.g., $V_\pi(s)$) recursively in terms of itself

02

Turn into iterative algorithm

Replace mathematical equality ($=$) with assignment operator ($\leftarrow$)

03

Initialize and iterate

Start with arbitrary values, repeat updates until convergence

This recipe works for search DP ($\text{FutureCost}(s)$), policy evaluation ($V_\pi(s)$), and value iteration ($V_{\text{opt}}(s)$)! 🎉

Comparison with Search Problems

Transitions	Deterministic	Stochastic
Solution	Path (sequence of actions)	Policy (state → action)
Evaluation	Cost (sum of edge costs)	Expected utility
Algorithm	DP (FutureCost recurrence)	Value iteration

MDPs **generalize** search problems by introducing randomness. The algorithmic approach remains the same! 🔗



Real-World Considerations 🌐

Model Uncertainty

What if we don't know $T(s, a, s')$ or $\text{Reward}(s, a, s')$? **Reinforcement learning** learns from experience! (Next lecture)

Large State Spaces

Real problems have millions/billions of states. Need **function approximation** and **deep learning** techniques.

Multiple Agents

What if other agents also make decisions? Enter **game theory** and **adversarial games** (upcoming lectures).

Key Takeaways 🎯

- **MDPs** formalize decision-making under uncertainty
  States, actions, transition probabilities, rewards

- Solutions are **policies**, not paths
  Policies handle all contingencies, evaluated by expected utility

- **Policy evaluation** computes value of a given policy
  Iteratively apply value-Q-value recurrences

- **Value iteration** finds optimal policy
  Maximize over actions instead of following fixed policy

- **Unified technique:** recurrences → algorithms
  Works for search DP, policy evaluation, and value iteration!

Looking Ahead: Reinforcement Learning 🚀

> **Challenge:** What if we don't know the MDP parameters T and Reward?

So far, we've assumed **complete knowledge** of the environment. Next lecture, we'll explore **reinforcement learning**:

- Learn optimal policies through **trial and error** 🎮
- Balance **exploration** vs. **exploitation** 🔍
- Model-free algorithms: **Q-learning**, **SARSA**, and beyond 📈

Get ready to tackle the unknown! See you next time. 👋

